

计算机网络实验报告

实验题目： HTTP、TCP 协议分析及基于 TCP 协议的 Socket 通信编程

一. 实验目的

- (1) 验证 HTTP 协议的报文结构，分析 HTTP 协议的工作方式。
- (2) 验证 TCP 协议的报文段结构，分析 TCP 协议的工作方式。
- (3) 编程实现基于 TCP 协议的 Socket 通信。

二. 实验环境与器材

实验环境：实验室机房

实验平台：

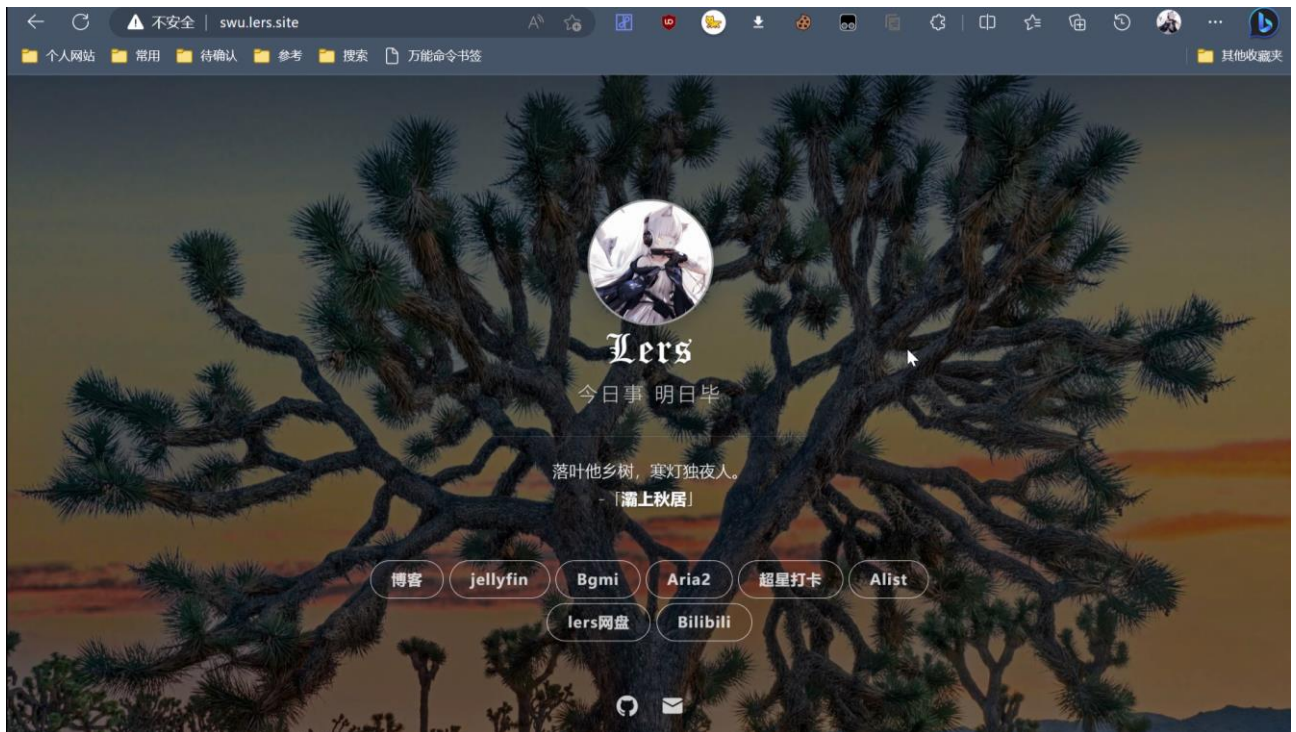
硬件：机房服务器、TL-XDR3010 易展版、拯救者 Y9000P2021

软件：VNC Server 6.11.0、VNC Viewer 7.0.1、Windows 11、Wireshark 3.6.6

三. 实验过程描述

(1) 访问任何一个 Web 站点，通过 Wireshark 软件捕获你的 Web 浏览器与上述站点之间交换的协议报文（如 HTTP 协议下的请求报文和响应报文、TCP 协议下的请求报文段和响应报文段），并结合课上积累的知识，尝试分析 HTTP 协议和 TCP 协议的工作方式。

访问站点:自己的导航页 (仅校园网内可以访问)



由于使用了 vnc 远程访问我的笔记本，所以捕获的时候有大量 udp 请求，过滤它们

```
(! (ip.src == 172.18.6.192)) && ! (ip.dst == 172.18.6.192)
```

然后就可以很清楚的看到与服务器的数据交换

No.	Time	Source	Destination	Protocol	Length	Info
4685	5.727107	192.168.0.104	192.168.0.120	HTTP	565	GET / HTTP/1.1
4688	5.728486	192.168.0.120	192.168.0.104	HTTP	556	HTTP/1.1 200 OK (text/html)
4695	5.768021	192.168.0.104	192.168.0.120	HTTP	490	GET /assets/css/onlinewebfonts.c
4702	5.769632	192.168.0.120	192.168.0.104	HTTP	1009	HTTP/1.1 200 OK (text/css)
4703	5.770392	192.168.0.104	192.168.0.120	HTTP	479	GET /assets/css/vno.css HTTP/1.1
4704	5.770689	192.168.0.104	192.168.0.120	HTTP	484	GET /assets/css/iconfont.css HT
4735	5.774579	192.168.0.120	192.168.0.104	HTTP	1197	HTTP/1.1 200 OK (text/css)
4809	5.778040	192.168.0.120	192.168.0.104	HTTP	1182	HTTP/1.1 200 OK (text/css)
4815	5.807168	192.168.0.104	192.168.0.120	HTTP	469	GET /assets/js/jquery.min.js HT
4816	5.808126	192.168.0.104	192.168.0.120	HTTP	463	GET /assets/js/main.js HTTP/1.1
4843	5.809232	192.168.0.104	192.168.0.120	HTTP	486	GET /assets/json/images.json?cb=
4874	5.809739	192.168.0.120	192.168.0.104	HTTP	947	HTTP/1.1 200 OK (application/ja
4905	5.810075	192.168.0.120	192.168.0.104	HTTP/J...	1035	HTTP/1.1 200 OK , JavaScript Obj
4918	5.810208	192.168.0.120	192.168.0.104	HTTP	1105	HTTP/1.1 200 OK (application/ja
4922	5.811727	192.168.0.104	192.168.0.120	HTTP	515	GET /assets/img/logo.png HTTP/1.
5317	5.824900	192.168.0.120	192.168.0.104	HTTP	271	HTTP/1.1 200 OK (PNG)

▼ Frame 4685: 565 bytes on wire (4520 bits), 565 bytes captured (4520 bits) on interface \Device\NPF_{03B6E402-A465-4B99-A1E5-01273674F1D2}

Interface id: 0 (\Device\NPF_{03B6E402-A465-4B99-A1E5-01273674F1D2})

Encapsulation type: Ethernet (1)

Arrival Time: Mar 30, 2023 10:55:36.126027000 中国标准时间

[Time shift for this packet: 0.000000000 seconds]

Epoch Time: 1680144936.126027000 seconds

解析其中的具体报文：

【请求报文】

以客户端发送的第一次请求为例

```
GET / HTTP/1.1
Host: swu.lers.site
Connection: keep-alive
Cache-Control: no-cache
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/111.0.0.0 Safari/537.36 Edg/111.0.1661.54
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;
q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Accept-Encoding: gzip, deflate
```

第一行：HTTP 协议

GET / HTTP/1.1

GET 表示请求方式，请求报文不一定使用 GET，也可以是 POST。后面的 HTTP/1.1 指的是协议版本。

第二行：请求对象主机

Host: swu.lers.site

表示网页来源于域名 swu.lers.site，端口没有指定，默认 80 端口。

第三行：连接模式

Connection: keep-alive

在 HTTP/1.1 中默认就是 keep-alive，除非显式的使用 close。意味着客户端和服务端约定在服务器返回响应报文之后不关闭 TCP 连接，并且在客户端接收响应报文之后也不关闭。

第四行：缓存控制

Cache-Control: no-cache

Cache-Control 允许客户端定义某一个响应资源应该何时、如何被缓存，以及缓存多久。这里不缓存接下来的响应报文。

第五行：协议升级

Upgrade-Insecure-Requests: 1

在一些 https 页面中调用 http 资源会导致浏览器抛出一些错误，为了解决这个问题，chrome 核心的浏览器会在 http 请求中加入这句话，询问服务器是否可以把这些 http 请求换成 https 请求，如果可以，服务器会返回 Content-Security-Policy: upgrade-insecure-requests，告诉浏览器可以升级这些 http 请求。

第六行：用户代理

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/111.0.0.0 Safari/537.36 Edg/111.0.1661.54

这是一个特殊字符串头，使得服务器能够识别客户使用的操作系统及版本、CPU 类型、浏览器及版本、浏览器渲染引擎、浏览器语言、浏览器插件等等。

第七行：客户端期望类型

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

描述客户端希望接受到的响应报文中的 body 数据类型。

第八行：客户端浏览器支持的语言

Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

第九行：客户端浏览器支持的编码格式

Accept-Encoding: gzip, deflate

【响应报文】

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Thu, 30 Mar 2023 02:55:36 GMT
Content-Type: text/html
Last-Modified: Tue, 28 Feb 2023 05:19:47 GMT
Transfer-Encoding: chunked
Connection: keep-alive
ETag: W/"63fd8ef3-186f"
Content-Encoding: gzip
```

第一行为状态行，200 代表成功访问，其他行都是一些信息。

(2) 验证 TCP 协议的报文段结构，分析 TCP 协议的工作方式。

```
Transmission Control Protocol, Src Port: 1936, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 1936
  Destination Port: 80
  [Stream index: 6]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1255714468
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
  > Flags: 0x002 (SYN)
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0x8257 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale
  > [Timestamps]
```

TCP 报文结构如上，源端口、目的端口、序列号、确认号、首部长度的、保留位、标志位、窗口大小、校验和、紧急指针、选项、数据

TCP 协议的工作方式如下：

1. 连接建立：TCP 连接由三个阶段组成，分别是建立连接、数据传输和连接终止。建立连接的过程包括“三次握手”，即客户端向服务器发送 SYN 报文段，服务器回应 SYN+ACK 报文段，客户端再回应 ACK 报文段，建立连接后，客户端和服务器之间才能进行数据传输。

TCP	66	1936 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1
TCP	66	80 → 1936 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM=1...
TCP	54	1936 → 80 [ACK] Seq=1 Ack=1 Win=2097920 Len=0

2. 数据传输：TCP 协议通过序列号和确认号来保证数据的可靠传输。每个 TCP 报文段中都有一个序列号

```
Transmission Control Protocol, Src Port: 1936, Dst Port: 80, Seq: 1, Ack: 1
  Source Port: 1936
  Destination Port: 80
  [Stream index: 6]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 511]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 1255714469
  [Next Sequence Number: 512 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 932935081
  0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x018 (PSH, ACK)
  Window: 8195
  [Calculated window size: 2097920]
  [Window size scaling factor: 256]
  Checksum: 0x844a [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > [Timestamps]
  > [SEQ/ACK analysis]
  TCP payload (511 bytes)
  > Hypertext Transfer Protocol
```

(3) 编程实现基于 TCP 协议的 Socket 通信。

首先服务器进入监听状态：

```
C:\Users\lenovo\AppData\Local\Programs\Python\Python39\python.exe D:/作业/计算机网络/实验/第四次网络实验课(实验指导)_2023.03.23/python/server.py
服务端开始监听
|
```

然后启动客户端发送消息然后接受反馈下线：

```
C:\Users\lenovo\AppData\Local\Programs\Python\Python39\python.exe D:/作业/计算机网络/实验/第四次网络实验课(实验指导)_2023.03.23/python/client.py
成功连接服务器: ('127.0.0.1', 8081)
请输入要发送的数据: 测试
【服务器消息】('127.0.0.1', 8081) : 收到
I
进程已结束,退出代码0
```

服务端接收到消息：

```
C:\Users\lenovo\AppData\Local\Programs\Python\Python39\python.exe D:/作业/计算机网络/实验/第四次网络实验课(实验指导)_2023.03.23/python/server.py
服务端开始监听
【新客户端】: ('127.0.0.1', 5154) 已连接
【客户端消息】('127.0.0.1', 5154) : 测试
客户端: ('127.0.0.1', 5154) 已下线
```

服务端实现:

```
import socket
import threading

def recv(client_socket, ip_port):
    while True:
        client_text = client_socket.recv(1024)
        # 如果接收的消息长度不为 0, 则将其解码输出
        if client_text:
            print("[客户端消息]", ip_port, ":", client_text.decode("gbk"))
            # 给客户端响应
            client_socket.send("收到\n".encode("gbk"))
            # 当客户端断开连接时, 会一直发送''空字符串, 所以长度为 0 已下线
        else:
            print("客户端:", ip_port, "已下线")
            client_socket.close()
            break

def StartSocket(ip, port):
    tcp_server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    address = (str(ip), port)
    tcp_server.bind(address)
    # 切换至监听状态, 最大连接数 128
    tcp_server.listen(128)
    print("服务端开始监听")
    while True:
        # 等待客户端连接
        client_socket, ip_port = tcp_server.accept()
        print("[新客户]:", ip_port, "已连接")
        # 有客户端连接后, 创建一个线程将客户端套接字, IP 端口传入 recv 函数,
        t1 = threading.Thread(target=recv, args=(client_socket, ip_port))
        # 设置线程守护
        t1.setDaemon(True)
        # 启动线程
        t1.start()

if __name__ == "__main__":
    StartSocket("0.0.0.0", 8081)
```

客户端实现:

```
import socket

def main():
    # 1. 创建 tcp 的套接字
    tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

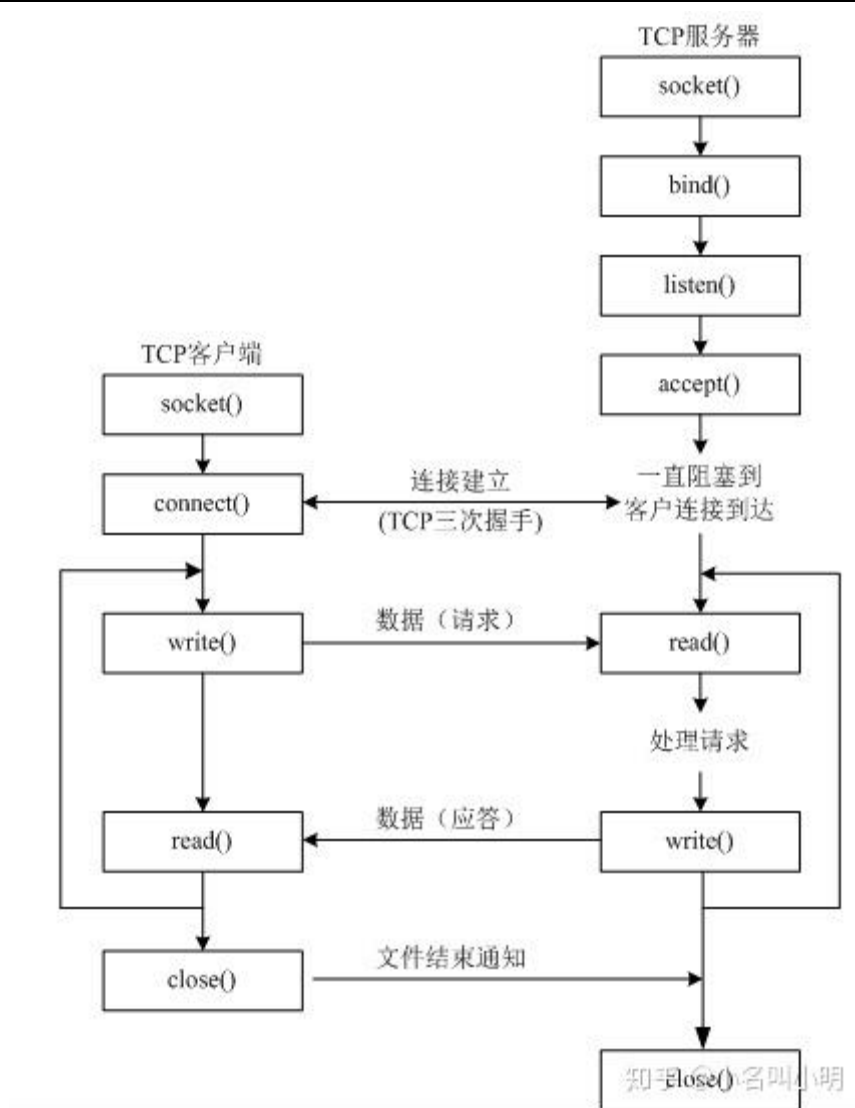


```

# 2. 链接服务器
server_addr = ("127.0.0.1", 8081)
tcp_socket.connect(server_addr)
print("成功连接服务器:", server_addr)
while True:
    # 3. 发送数据/接收数据
    send_data = input("请输入要发送的数据:")
    tcp_socket.send(send_data.encode("gbk"))
    server_text = tcp_socket.recv(1024)
    if server_text:
        print("[服务器消息]", server_addr, ":", server_text.decode("gbk"))
        break
# 4. 关闭套接字
tcp_socket.close()

if __name__ == "__main__":
    main()

```



这张图可以清晰明了的解释 socket 服务端和客户端通讯的一个过程。

四. 实验结果分析

这次实验包括对 tcp 协议通讯的抓包解析和实现简单的 socket 通讯服务器。在抓包的过程中我第一次尝试抓的是校园网的统一身份认证系统登录页面，然而在 wireshark 上找到的 tcp 通讯内容不可读，在搜索引擎的加持下，我发现了原因所在。校园网的统一身份认证系统使用了 https，也是就说，这个 tcp 通讯是加密的，所以不可读，但是在浏览器中的“网络”中可以看到解密后的内容，这就和 tls 协议有关系了，具体参考了[这篇文章](#)，故改为抓我的校园网内导航页，是一个 http，没有加密。

在实现 socket 服务器的时候，非常的顺利，理解了原理之后很快就写出了代码，关于 python 的多线程用法参考了 CSDN 上的一篇文章，写的很通俗易懂。

五. 实验相关问题回答

在上述实验的基础上，请回答以下问题：

1. 请尝试分析应用层协议的通用设计思路。

1. 定义协议数据单元（PDU）：应用层协议需要定义一种特定的数据格式，称为协议数据单元（PDU），用于在应用程序之间传输数据。PDU 的设计应该满足特定的应用需求和网络环境。
2. 定义协议的操作和消息：应用层协议需要定义一组操作和消息，用于描述应用程序之间的通信方式。例如，HTTP 协议定义了 GET、POST、PUT、DELETE 等请求方法，以及对应的响应消息。
3. 定义协议的状态管理：应用层协议需要定义一些状态变量，用于跟踪应用程序之间的交互过程。例如，HTTP 协议使用 Cookie 机制来跟踪客户端的状态。
4. 定义协议的错误处理机制：应用层协议需要定义一套错误处理机制，用于在通信过程中发现错误时进行恢复。例如，HTTP 协议定义了一套状态码和错误响应消息，用于描述错误的类型和原因。
5. 考虑协议的扩展性和兼容性：应用层协议应该考虑到未来的扩展需求，同时保持向后兼容性。例如，HTTP 协议支持通过协商扩展来添加新的功能，而保持与旧版本的兼容性。

参考对象：

[手把手教你实现自定义的应用层协议 - 简书 \(jianshu.com\)](#)

[如何设计应用层协议\(草稿\)_Co_zy 的博客-CSDN 博客](#)

[ChatGPT](#)